

Red Social Básica

Documentación del Sistema

Equipo 12

Lara Vera Iván Soria Cabrera Andres Serrano Arriaga Sahian

Desarrollo Web – FES Acatlán, UNAM

Junio 2026

Índice

1. Introducción	2
1.1. Objetivo general	2
1.2. Objetivos específicos	2
1.3. Alcance	2
2. Marco Teórico (Visión del Cliente)	2
3. Descripción del sistema	2
3.1. Perspectiva	2
3.2. Funciones principales	2
3.3. Tipos de usuario	3
3.4. Tecnologías	3
4. Requisitos del sistema	3
4.1. Requerimientos funcionales	3
4.2. Requerimientos no funcionales	4
5. Casos de uso	4
5.1. CU-01: Registro	4
5.2. CU-02: Login	4
5.3. CU-03: Publicar	4
5.4. CU-04: Like (toggle)	5
5.5. CU-05: Comentar	5
5.6. CU-06: Editar perfil	5
5.7. Diagrama Entidad-Relación	5
6. Pantallas principales	6
6.1. Componentes compartidos	6
7. Documentación técnica	6
7.1. Estructura del proyecto	6
7.2. Stack	7
7.3. Comandos	7
7.4. Arquitectura MVC	7
8. Conclusiones	7

1. Introducción

1.1. Objetivo general

Desarrollar una aplicación web que permita la interacción entre usuarios mediante publicaciones, comentarios y reacciones en un entorno social básico.

1.2. Objetivos específicos

1. Permitir a los usuarios crear publicaciones con texto e imágenes.
2. Implementar interacción mediante “likes” (reacciones tipo toggle).
3. Permitir comentarios en publicaciones.
4. Mostrar un feed ordenado cronológicamente.
5. Gestionar perfiles básicos de usuario.

1.3. Alcance

Incluye: registro y login, feed cronológico, publicaciones con imágenes, likes toggle, comentarios, perfiles editables.

Excluye: mensajería, seguidores, notificaciones, panel admin, búsqueda.

2. Marco Teórico (Visión del Cliente)

El cliente desea un prototipo funcional para validar una idea de negocio, con la visión de escalar a una plataforma social más grande. El prototipo implementa las funcionalidades básicas (publicaciones, likes, comentarios, perfiles) usando un stack contenerizado con Docker que permite despliegue reproducible en cualquier entorno.

3. Descripción del sistema

3.1. Perspectiva

Aplicación web monolítica con arquitectura Modelo-Vista-Controlador (MVC). Servidor Express + vistas EJS renderizadas del lado del servidor. Interacciones asíncronas mediante fetch API. Contenerizada con Docker Compose (MySQL + API).

3.2. Funciones principales

- **Autenticación:** registro, login con sesiones en MySQL, logout.
- **Publicaciones:** crear con texto (máx. 5000 caracteres) e imagen opcional.
- **Feed:** listado cronológico inverso de todas las publicaciones.
- **Likes:** toggle (dar/quitar) sobre publicaciones.
- **Comentarios:** crear y visualizar en publicaciones.

- **Perfil:** ver y editar datos personales (nombre, bio, foto).

3.3. Tipos de usuario

- **Visitante:** accede a login y registro. Redirigido si intenta rutas protegidas.
- **Autenticado:** acceso completo al feed, publicar, comentar, reaccionar, editar perfil.

3.4. Tecnologías

Tecnología	Versión	Uso
Node.js	20 Alpine	Entorno de ejecución
Express	5.2.1	Framework web
EJS	5.0.2	Motor de plantillas
MySQL	8.0	Base de datos relacional
mysql2	3.22.3	Driver MySQL para Node.js
bcrypt	6.0.0	Hashing de contraseñas
express-session + mysql-session	1.19 / 3.0.3	Sesiones en MySQL
Zod	4.4.3	Validación de datos
Multer	2.1.1	Subida de imágenes
Docker + Compose	27+	Contenerización

4. Requisitos del sistema

4.1. Requerimientos funcionales

- RF1: Publicaciones.** Crear publicaciones con texto (máx. 5000) e imagen opcional (JPEG, PNG, GIF, WebP; 5 MB máx.). Feed ordenado por fecha descendente.
- RF1: Likes.** Toggle like/unlike sobre publicaciones. Una reacción por usuario por publicación (UNIQUE en BD).
- RF1: Comentarios.** Escribir comentarios en publicaciones (máx. 2000 caracteres). Orden cronológico ascendente.
- RF1: Autenticación.** Registro con nombre, email y contraseña (≥ 8). Login con email y contraseña. Contraseñas hasheadas con bcrypt.
- RF1: Feed cronológico.** Publicaciones ordenadas de más reciente a más antigua.
- RF1: Perfil de usuario.** Ver y editar perfil (nombre, biografía, foto). Ver perfil de otros usuarios.
- RF1: Subida de imágenes.** Imágenes en publicaciones y foto de perfil. Formatos: JPEG, PNG, GIF, WebP.
- RF1: Control de acceso.** Middleware requireAuth protege rutas de feed, perfil, publicaciones, comentarios y reacciones.
- RF1: Validación de datos.** Zod valida todas las entradas. Errores en español.

4.2. Requerimientos no funcionales

RNF2: Escalabilidad. Contenedores Docker separados (MySQL + API) permiten escalar horizontalmente.

RNF2: UX amigable. Feedback visual: estados de carga en botones, alertas de error/éxito, likes sin recarga.

RNF2: Seguridad. bcrypt para contraseñas. Cookies httpOnly. Mensajes genéricos en login. Logout destruye sesión.

RNF2: Despliegue reproducible. Un comando: `docker compose up -build`. Incluye BD, esquema, dependencias.

RNF2: Optimización. JOINS y subconsultas de conteo evitan N+1 queries.

RNF2: Persistencia. Volúmenes Docker para datos MySQL e imágenes subidas.

5. Casos de uso

5.1. CU-01: Registro

Actor: Visitante. **Precondición:** Sin sesión activa.

1. Accede a `/register`. Ingresa nombre, email y contraseña.
2. El sistema valida con Zod, verifica email único, hashlea con bcrypt y guarda en BD.
3. Respuesta `{ok: true}` (201). Redirige a `/login`.

Excepción: Email duplicado $\rightarrow$ 400.

5.2. CU-02: Login

Actor: Visitante. **Precondición:** Sin sesión.

1. Accede a `/login`. Ingresa email y contraseña.
2. El sistema busca por email, compara hash bcrypt, crea sesión en MySQL.
3. Respuesta `{ok: true}`. Redirige a `/feed`.

Excepción: Credenciales incorrectas $\rightarrow$ 401 (mensaje genérico).

5.3. CU-03: Publicar

Actor: Autenticado. **Precondición:** En `/feed`.

1. Escribe contenido en textarea. Opcional: selecciona imagen.
2. Envía FormData vía fetch POST a `/feed`.
3. Multer guarda imagen, Zod valida texto, modelo inserta en BD.
4. Respuesta `{ok: true, publicacion}` (201). Recarga feed.

5.4. CU-04: Like (toggle)

Actor: Autenticado. **Precondición:** Publicación existente.

1. Clic en botón like. Fetch POST a `/reaccion/:id/toggle`.
2. Si no existe reacción: INSERT. Si ya existe: DELETE (por UNIQUE).
3. Respuesta `{reaccionado: bool, total: int}`. Ícono y contador se actualizan sin recarga.

5.5. CU-05: Comentar

Actor: Autenticado. **Precondición:** Publicación existente.

1. Expande sección de comentarios. Escribe texto.
2. Fetch POST a `/comentario/:id` con JSON.
3. Zod valida (máx. 2000 caracteres). Modelo inserta en BD.
4. Respuesta `{ok: true, comentario}` (201). Recarga página.

5.6. CU-06: Editar perfil

Actor: Autenticado. **Precondición:** En `/perfil`.

1. Ve formulario con datos precargados. Modifica nombre, bio o foto.
2. Envía FormData vía fetch PUT a `/perfil`.
3. Multer guarda nueva foto, modelo actualiza solo campos modificados.
4. Respuesta `{ok: true, usuario}`. Sesión se actualiza. Redirige al perfil.

5.7. Diagrama Entidad-Relación

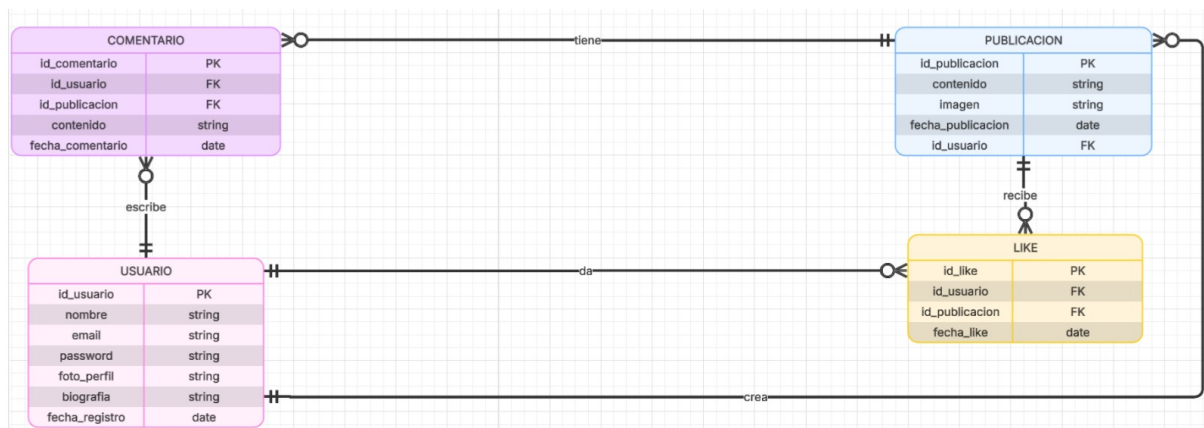


Figura 1: Diagrama Entidad-Relación de la Red Social Básica

6. Pantallas principales

Pantalla	Descripción
Login	Formulario con email y contraseña. Fetch POST a /login en JSON. Errores inline. Si ya tiene sesión, redirige al feed.
Registro	Campos: nombre, email, contraseña, confirmación. Validación client-side (mín. 8 caracteres, coincidencia). Registro exitoso redirige al login.
Feed	Caja de publicación (textarea + botón de imagen). Lista de tarjetas con: avatar, nombre, fecha, contenido, imagen, botón like (/), comentarios (toggle), sección de comentarios colapsable, eliminar (solo autor). Likes se actualizan sin recarga.
Perfil	Cabecera con foto, nombre, email, biografía, fecha de registro. Botón “EDITAR PERFIL” (solo perfil propio). Listado de publicaciones del usuario.
Editar perfil	Formulario con datos precargados. Vista previa de foto. Campos: nueva foto, nombre, biografía. Guardar (PUT con FormData) o Cancelar.

6.1. Componentes compartidos

- `head.ejs`: Meta tags, tipografías (Inter), CSS global (nav, cards, botones, formularios, comentarios, alertas). Script de verificación de sesión al recuperar visibilidad de pestaña.
- `navbar.ejs`: Logo “CONECTA2”, enlaces INICIO y MI PERFIL. Si hay sesión: avatar + nombre + botón SALIR. Si no: botón INICIAR SESIÓN.

7. Documentación técnica

7.1. Estructura del proyecto

```
Proyecto/  
  10-autenticacion.js      # Entry point Express  
  controllers/             # auth, publicacion, reaccion,  
                           # comentario, perfil  
  database/init.sql        # Esquema MySQL  
  docker-compose.yml       # Servicios: mysql-app + api  
  middlewares/             # auth, upload (multer), validate (zod)  
  models/mysql/            # usuarios, publicaciones,  
                           # reacciones, comentarios, db.js  
  routes/                  # auth, publicacion, reaccion,  
                           # comentario, perfil  
  schemas/                 # auth, publicacion, comentario, perfil  
  uploads/                 # Imágenes (volumen Docker)
```

```

views/                # feed, login, register,
  partials/           #  perfil, editar_perfil
                        #  head.ejs, navbar.ejs
api/Dockerfile        # Imagen Node.js
package.json
docker-compose.yml

```

7.2. Stack

Node.js 20 + Express 5 + EJS 5 + MySQL 8 + mysql2 3 + bcrypt 6 + express-session + Zod 4 + Multer 2 + Docker Compose.

7.3. Comandos

Comando	Descripción
<code>docker compose up -build</code>	Construye imagen y levanta ambos contenedores.
<code>docker compose down -v</code>	Detiene y elimina volúmenes (BD limpia).
<code>docker logs</code>	Logs de la API.
<code>proyectodesarrolloweb-api-1</code>	
<code>docker exec conecta2 mysql</code>	Consola MySQL.
<code>-u root -padmin</code>	

7.4. Arquitectura MVC

- **Modelos:** consultas SQL parametrizadas con `mysql2/promise`. Pool de conexiones en `db.js`.
- **Vistas:** plantillas EJS con `partials` reutilizables (`head`, `navbar`).
- **Controladores:** clases con métodos estáticos asíncronos. Responden con `res.render()` (HTML) o `res.json()` (API).
- **Rutas:** definen endpoints y encadenan `middlewares`.
- **Middlewares:** autenticación, upload, validación, control de caché, variables globales de vista.
- **Schemas:** definiciones `Zod` para cada operación con entrada de usuario.

8. Conclusiones

La Red Social Básica implementa todas las funcionalidades requeridas: publicaciones con imágenes, likes toggle, comentarios, feed cronológico y perfiles editables. Se aplicaron buenas prácticas de ingeniería de software:

- **MVC:** separación clara de responsabilidades entre modelos, vistas, controladores, rutas, `middlewares` y `schemas`.

- **Seguridad:** bcrypt para contraseñas, cookies httpOnly, Zod para validación, mensajes genéricos en login.
- **Docker:** entorno reproducible con un solo comando. Volúmenes persistentes para BD e imágenes.
- **Optimización:** JOINS y subconsultas evitan N+1 queries. Caché deshabilitada en rutas protegidas.
- **UX:** interacciones asíncronas (likes sin recarga), estados de carga en botones, alertas de error/éxito.

El prototipo cumple con la visión del cliente y sienta las bases para escalar a funcionalidades adicionales (mensajería, seguidores, notificaciones) sin reestructurar la arquitectura.